POC 1 - Moteur

Appel d'offres ARES · Java

Contexte

Le moteur est l'arbitre du jeu : il applique les règles, joue un match tour par tour et produit le log d'événements.

Positions continues (`x` , `y` en flottant), orientation en radians. `MOVE` et `SHOOT` prennent un angle. Le robot avance de `move_speed` dans la direction de l'angle.

État initial (`poc1-init.json`) :

```
{
  "world": { "w": 100, "h": 100 },
  "move_speed": 2.0,
  "robots": [
    { "id": "A", "x": 40.0, "y": 50.0, "heading": 0.0, "energy": 20 },
    { "id": "B", "x": 60.0, "y": 50.0, "heading": 3.14, "energy": 20 }
  ]
}
```

Les actions, elles, ne sont pas fournies : elles sont **saisies au clavier**, une par ligne, pendant que le moteur tourne. Format d'une ligne : l'identifiant du robot, l'action, et ses paramètres éventuels.

```
A MOVE 0.0
B SHOOT 3.14
A MINE
```

À chaque ligne saisie, le moteur applique l'action et écrit l'événement correspondant sur sa sortie.

Log attendu (extrait, pour les saisies ci-dessus) :

```
{ "seq": 0, "type": "ROBOT_MOVED", "robot": "A", "x": 42.0, "y": 50.0 }
{ "seq": 1, "type": "ROBOT_SHOT", "robot": "B", "angle": 3.14, "hit": "A" }
{ "seq": 2, "type": "ROBOT_DAMAGED", "robot": "A", "hp": 75 }
{ "seq": 3, "type": "ACTION_REJECTED", "robot": "A", "reason": "NO_CRYSTAL" }
```

À faire

Partie 1 : Moteur

Écrire un petit moteur qui charge l'état initial, lit les actions saisies au clavier, les applique (énergie, collisions) et produit le log. Actions : `MOVE` , `MINE` , `SHOOT` , `WAIT` .

Partie 2 : Extension

Ajouter une cinquième action, `TELEPORT` (téléporte le robot vers une position donnée), dans un nouveau commit.

Rendu

- Le code, avec `TELEPORT` en commit séparé.
- Un README court : ce que touche ce commit, ce qui a été difficile.

Pas de réseau, pas de base, pas de framework.